

suspend() and resume() method

08 March 2026 10:26

How to suspend and resume a thread?

```
public void suspend()  
public void resume()
```

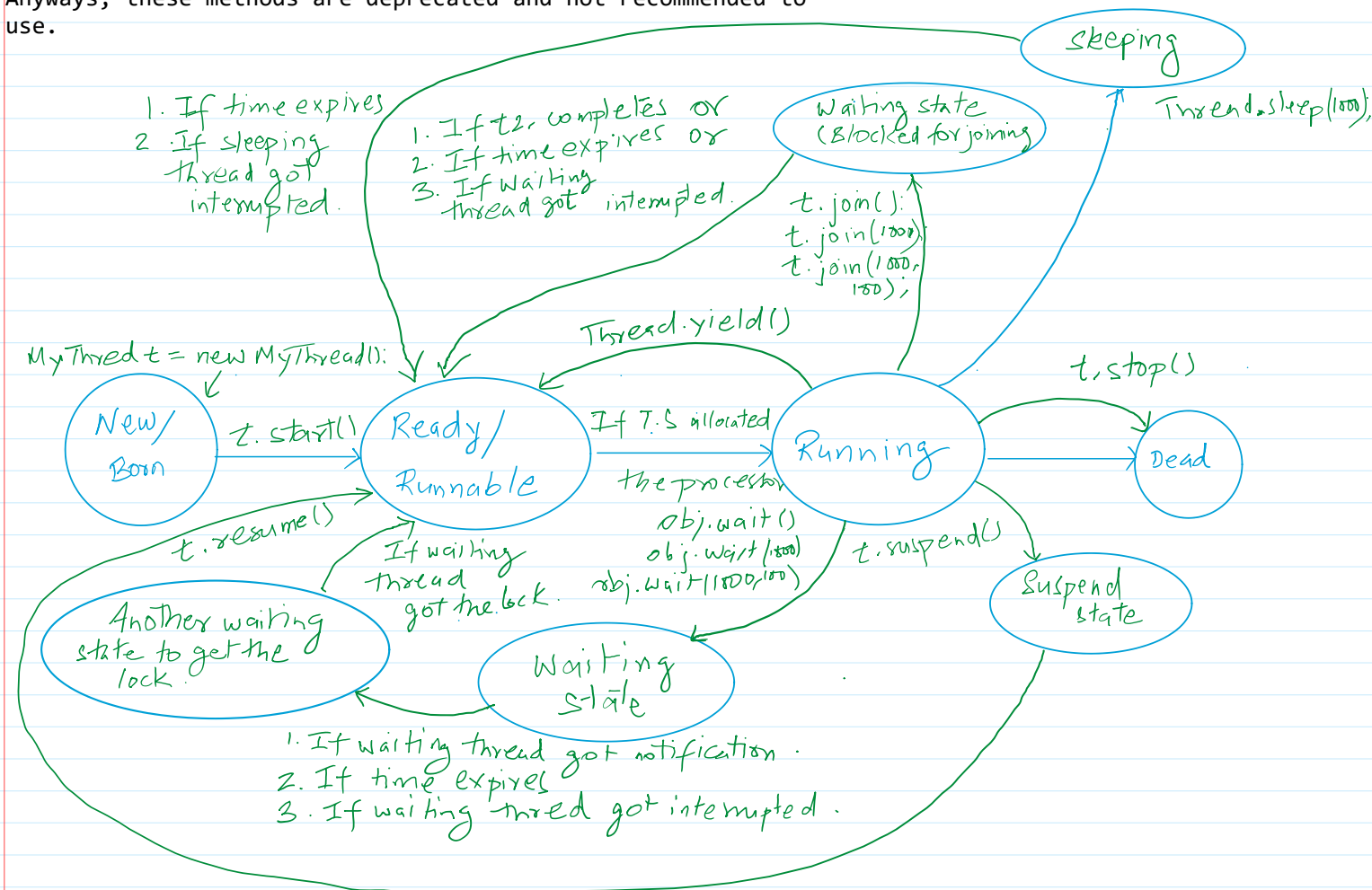
`t.suspend();`

`t.resume();`

We can suspend a thread by using `suspend()` method of Thread class, then immediately the thread will be entered into suspended state.

We can resume a suspended thread by using `resume()` method of Thread class. Then suspended thread can continue its execution.

Anyways, these methods are deprecated and not recommended to use.



Why `suspend()` and `resume()` are Deprecated?

These methods are unsafe because they may cause deadlock.

Example Problem:

If a thread holds a lock on a resource and gets suspended.

* Other threads cannot access that resource.

* The program may freeze.

* Other threads cannot access that resource.

* The program may freeze.

Because of this issue, Java deprecated these methods starting from Java 1.2.

Modern Alternative (Recommended).

Instead of `suspend()` and `resume()`, developers use:

* `wait()` * `notify()` * `notifyAll()` * volatile flags.

```
package MyThreads.SuspendAndResume;
public class MyThread extends Thread{

    @Override
    public void run() {
        for (int i = 0; i < 10; i++) {
            System.out.println("Thread running: "+i);
            try {
                Thread.sleep(2000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}
```

```
package MyThreads.SuspendAndResume;
public class SuspendResumeDemo {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        t1.start();
        try {
            Thread.sleep(2000);
            System.out.println("Thread Suspended");
            t1.suspend();
            Thread.sleep(3000);
            System.out.println("Thread resume");
            t1.resume();
            for(int i = 0; i < 10; i++){
                System.out.println("Main Thread: "+ i);
            }
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

Output:

Thread running: 0

Thread Suspended

Exception in thread "main" java.lang.UnsupportedOperationException

because modern JVM is not supporting `suspend()` method.

```

        at java.base/java.lang.Thread.suspend(Thread.java:1826)
        at
MyThreads.SuspendAndResume.SuspendResumeDemo.main(SuspendResumeDemo.java:11)
Thread running: 1
Thread running: 2
Thread running: 3
Thread running: 4
Thread running: 5
Thread running: 6
Thread running: 7
Thread running: 8
Thread running: 9

```

By using modern alternative we can avoid deadlock and UnsupportedOperationException also

```

package MyThreads.SuspendAndResume;
public class MyThread extends Thread {
    volatile boolean suspended = false;
    public void suspendThread() {
        suspended = true;
    }
    public void resumeThread() {
        suspended = false;
    }
    @Override
    public void run() {
        try {
            for (int i = 0; i < 10; i++) {
                while (suspended) {
                    Thread.sleep(100);
                }
                System.out.println("Thread running: " + i);
                Thread.sleep(2000);
            }
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}

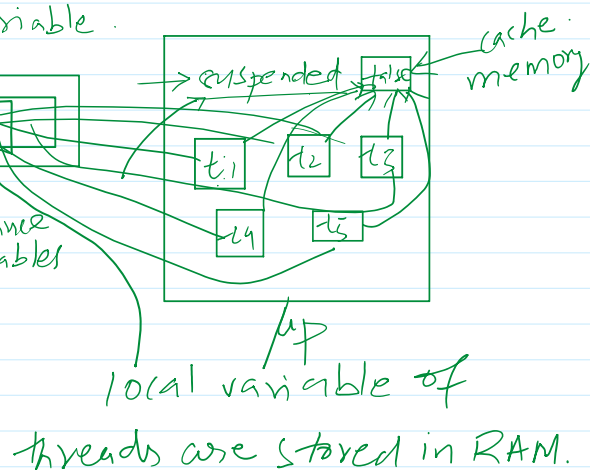
```

Ensures visibility of variable across all threads.

flag variable.

RAM

static & instance variables



```

package MyThreads.SuspendAndResume;
public class SuspendResumeDemo {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        t1.start();
        try {
            Thread.sleep(2000);
            System.out.println("Thread Suspended");
            t1.suspendThread();
            Thread.sleep(3000);

            for(int i = 0; i < 10; i++){
                System.out.println("Main Thread: " + i);
            }
            System.out.println("Thread resume");
            t1.resumeThread();
        }
    }
}

```

```
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

Output:

```
Thread running: 0  
Thread Suspended  
Main Thread: 0  
Main Thread: 1  
Main Thread: 2  
Main Thread: 3  
Main Thread: 4  
Main Thread: 5  
Main Thread: 6  
Main Thread: 7  
Main Thread: 8  
Main Thread: 9  
Thread resume  
Thread running: 1  
Thread running: 2  
Thread running: 3  
Thread running: 4  
Thread running: 5  
Thread running: 6  
Thread running: 7  
Thread running: 8  
Thread running: 9
```